

Exercise 3: Dynamically Create Components

Interfaces 102: Building Dynamic Interfaces

Introduction	2
How Can I Practice?	2
Appian Version	2
Naming Conventions	2
Save Often	2
Additional Resources	2
Use a!forEach() to Create Cards	3
Populate Cards with Placeholder Data	4
Replace Place Holder Data using fv!item	5
Example Interface Expression	7

Notice of Rights

This document was created by Appian Corporation, 7950 Jones Branch Dr, Tysons, Virginia 22102. Copyright 2026 by Appian Corporation. All rights reserved. Information in this document is subject to change. No part of this document may be reproduced or transmitted in any form by any means, without prior written permission of Appian Corporation. For more information on obtaining permission for reprints or excerpts, contact Appian Training at academyonline@appian.com.

Introduction

How Can I Practice?

You should practice in Appian Community Edition. Keep in mind that this is a community environment not suitable for production workloads or sensitive information.

Appian Version

Appian Community Edition is on the latest Appian version. If you are following the exercises from a previous Appian version, go to academy.appian.com to download the latest version.

Naming Conventions

When you register for Appian Community Edition, you gain access to a workspace, which is your personal area within the community environment. When your workspace has been assigned, you will receive a confirmation email with a workspace ID. Pre-loaded apps in your workspace include the workspace ID.

Use this workspace ID when creating new applications and objects. Otherwise, you could run into naming conflicts with objects created by other users. In the example below, the application prefix for the Acme Auto Solution app is W0000AS. So, the workspace ID is W0000, which would be included in any new app's prefix.

Save Often

Appian does not automatically save updates, so save your objects frequently.

Additional Resources

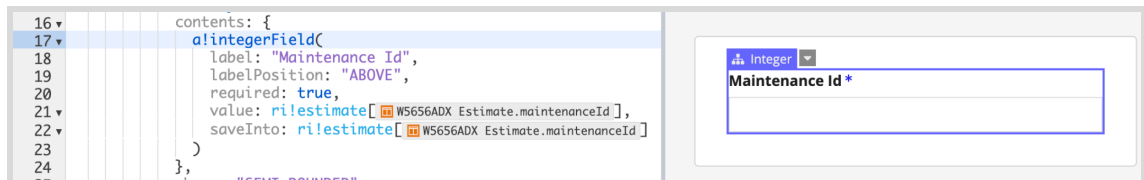
Appian provides a number of training resources for Appian developers. The following resources are particularly popular with our learners:

- [Academy Online](#) - Appian's online courses provide useful survey courses, step-by-step tutorials, and practice exercises. Explore these resources at your own pace. Survey courses will help you develop a better grasp of the topics you need to learn. Video and print tutorials will help you with getting hands-on experience with Appian.
- [Community Discussions for New Users](#) - Check out the **New to Appian** thread in Community. Join our community of experts to ask questions and find answers from past discussions.
- [Appian Documentation](#) - Appian's product documentation will provide you with an overview of key Appian features, newest release information, additional tutorials, and helpful patterns and recipes to implement in your app.

Use a!forEach() to Create Cards

In the last exercise, you queried the W#AA Maintenance record type and stores maintenance requests to the local variable **local!availableMaintenanceRequests**. Now, you will use **a!forEach()** to create cards for each item stored in that local variable.

1. Navigate to the a!integerField() currently in the interface used to collect the maintenance id. You can either scroll through the expression to find it, or click the component in the live preview.

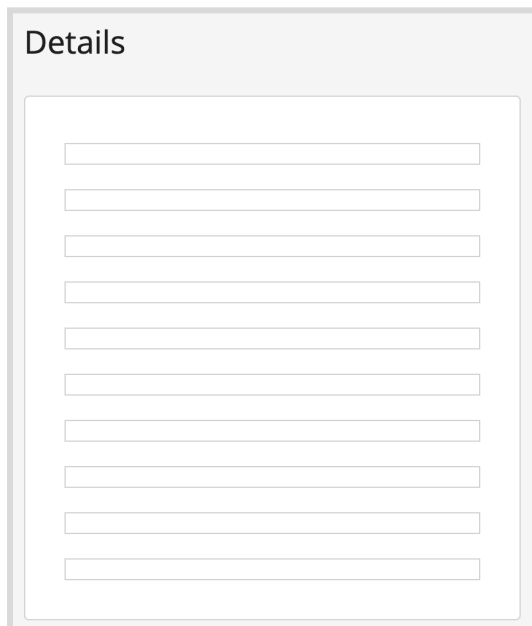


You will replace this integer field with an **a!cardGroupLayout()** and populate the card group layout with cards representing the maintenance requests.

2. Delete the integer field and replace it with the following:

```
a!cardGroupLayout(  
  cards: a!forEach(  
    items: local!availableMaintenanceRequests,  
    expression: a!cardLayout()  
  )  
)
```

Notice the **a!forEach()** function uses your local variable for the **items** parameter. This means that the **expression** parameter will be evaluated once for each item stored in that local variable. This will create 10 empty cards in your interface.



Populate Cards with Placeholder Data

It can sometimes be easier to set up the layout of interface components using placeholder data before populating with real record references and function variables.

Each card should include the vehicle image, year, make, and model as well as the maintenance issue. Ultimately, the layout of the cards is up to you, but a sample has been provided below.

```
a!cardLayout(  
  padding: "NONE",  
  contents: {  
    a!columnsLayout(  
      columns: {  
        a!columnLayout(  
          width: "NARROW",  
          contents: a!imageField(  
            labelPosition: "COLLAPSED",  
            images: a!documentImage(  
              document: a!EXAMPLE_DOCUMENT_IMAGE()  
            )  
          )  
        ),  
        a!columnLayout(  
          contents: {  
            a!headingField(  
              text: "Year Make Model",  
              size: "MEDIUM"  
            ),  
            a!richTextDisplayField(  
              labelPosition: "COLLAPSED",  
              value: "Maintenance issue ..."  
            )  
          }  
        )  
      }  
    )  
  }  
)
```

With this expression, your cards will resemble the following:



Replace Place Holder Data using fv!item

However you configured your cards, you will now need to populate them with real data. While inside an **a!forEach()** function, you can access the items being looped through using the function variable **fv!item**. In this case, that will be a W#AA Maintenance record and can be indexed into using standard record field references.

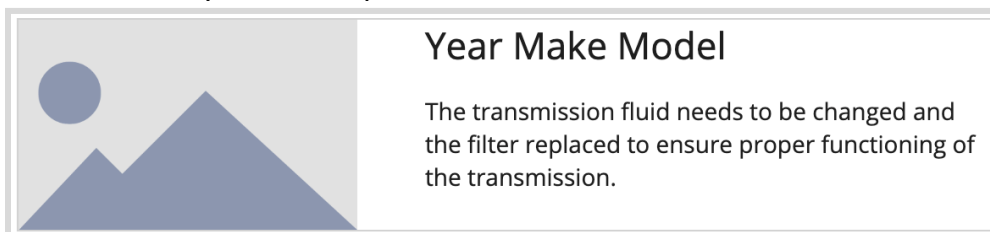
1. Wherever your placeholder text for the maintenance issue is in your expression, replace it with the actual issue field from the maintenance record type:

fv!item[W#AA Maintenance.issue]

```
a!richTextDisplayField(  
  labelPosition: "COLLAPSED",  
  value: fv!item[W5656AA01 Maintenance.issue ]  
)
```

If you used the example card configuration from above, this is within an **a!richTextDisplayField()**.

You should see your cards update with real record data.



2. Replace the placeholder data for the vehicle year, make and model. Remember that this data is stored in the related Vehicle record type and is accessed through the vehicle relationship.
 - **fv!item[W#AA Maintenance.vehicle.year]**
 - **fv!item[W#AA Maintenance.vehicle.make]**
 - **fv!item[W#AA Maintenance.vehicle.model]**

- You can use the operator **&** to concatenate these fields if needed.

```
a!headingField(  
  text: {  
    fv!item[ W5656AA01 Maintenance.vehicle.year ] & " " &  
    fv!item[ W5656AA01 Maintenance.vehicle.make ] & " " &  
    fv!item[ W5656AA01 Maintenance.vehicle.model ]  
  },  
  size: "MEDIUM"  
)
```

3. When configuring the **a!imageField()**, you should use an **if()** function to handle situations where the record does not have an associated image. Otherwise, you will get an error when the image field is null.

fv![W#AA Maintenance.vehicle.image]

```
a!imageField(  
  labelPosition: "COLLAPSED",  
  images: a!documentImage(  
    document: if(  
      condition: a!isNullOrEmpty(fv!item[ W5656AA01 Maintenance.vehicle.image ]),  
      valueIfTrue: a!EXAMPLE_DOCUMENT_IMAGE(),  
      valueIfFalse: fv!item[ W5656AA01 Maintenance.vehicle.image ]  
    )  
  )  
)
```

Your interface should now have cards displayed with information for each of the available maintenance requests. In the next exercise, you will make these cards selectable to save the user's selection.

Example Interface Expression

A full example expression of what your interface might look like at this stage is available below. Keep in mind that you will not be able to copy and paste this into your interface as record type field references are unique to each workspace.

```

a!localVariables(
  local!availableMaintenanceRequests:
    rule!W#ADX_getRequestedMaintenance(),
  a!wizardLayout(
    titleBar: a!headerTemplateFull(
      title: "Create Estimate",
      secondaryText: "Enter details for the estimate"
    ),
    isTitleBarFixed: true,
    showButtonDivider: true,
    backgroundColor: "#F6F7F9",
    steps: {
      a!wizardStep(
        label: "Details",
        contents: {
          a!cardLayout(
            contents: {
              a!cardGroupLayout(
                cards: a!forEach(
                  items: local!availableMaintenanceRequests,
                  expression: a!cardLayout(
                    padding: "NONE",
                    contents: {
                      a!columnsLayout(
                        columns: {
                          a!columnLayout(
                            width: "NARROW",
                            contents: a!imageField(
                              labelPosition: "COLLAPSED",
                              images: a!documentImage(
                                document: if(
                                  condition: a!isNullOrEmpty(
                                    fv![W#AA
                                      Maintenance.vehicle.image]
                                  ),
                                  valueIfTrue:
                                    a!EXAMPLE_DOCUMENT_IMAGE(),
                                  valueIfFalse:
                                    fv![W#AA
                                      Maintenance.vehicle.image]
                                )
                              )
                            )
                          )
                        )
                      ),
                      a!columnLayout(
                        contents: {
                          a!headingField(

```

```

        text: {
            fv!item[W#AA Maintenance.vehicle.year]
            & " " &
            fv!item[W#AA Maintenance.vehicle.make]
            & " " &
            fv!item[W#AA Maintenance.vehicle.model]
        },
        size: "MEDIUM"
    ),
    a!richTextDisplayField(
        labelPosition: "COLLAPSED",
        value: fv!item[W#AA Maintenance.issue]
    )
}
)
}
)
)
)
),
shape: "SEMI_ROUNDED",
padding: "MORE",
marginBelow: "STANDARD",
style: "NONE"
)
}
),
a!wizardStep(
    label: "Part",
    contents: {
        a!cardLayout(
            contents: {
                a!gridLayout(
                    label: "Part",
                    labelPosition: "COLLAPSED",
                    instructions: "Enter parts for the estimate",
                    height: "AUTO",
                    borderStyle: "LIGHT",
                    shadeAlternateRows: false,
                    rowHeader: 1,
                    headerCells: {
                        a!gridLayoutHeaderCell(label: "Name"),
                        a!gridLayoutHeaderCell(label: "Unit Cost"),
                        a!gridLayoutHeaderCell(label: "Quantity"),
                        a!gridLayoutHeaderCell()
                    },
                    columnConfigs: {
                        a!gridLayoutColumnConfig(width: "NARROW_PLUS"),
                        a!gridLayoutColumnConfig(width: "NARROW_PLUS"),
                        a!gridLayoutColumnConfig(width: "NARROW_PLUS"),
                        a!gridLayoutColumnConfig(width: "ICON")
                    }
                )
            }
        )
    }
)

```

```

    },
    rows: a!forEach(
        items: ri!estimate[W#ADX Estimate.part],
        expression: a!gridRowLayout(
            id: fv!index,
            contents: {
                a!textField(
                    value: fv!item[W#ADX Part.name],
                    saveInto: fv!item[W#ADX Part.name]
                ),
                a!floatingPointField(
                    value: fv!item[W#ADX Part.unitCost],
                    saveInto: fv!item[W#ADX Part.unitCost],
                ),
                a!integerField(
                    value: fv!item[W#ADX Part.quantity],
                    saveInto: fv!item[W#ADX Part.quantity]
                ),
                a!imageField(
                    size: "ICON",
                    images: a!documentImage(
                        document: a!iconIndicator(icon: "REMOVE"),
                        altText: "Remove Row" & char(32) & fv!index + 1,
                        link: a!dynamicLink(
                            value: fv!index,
                            saveInto: {
                                a!save(
                                    ri!estimate[W#ADX Estimate.part],
                                    remove(
                                        ri!estimate[W#ADX Estimate.part],
                                        save!value
                                    )
                                )
                            }
                        )
                    )
                )
            }
        )
    ),
    addRowLink: a!dynamicLink(
        label: "Add Row",
        value: W#ADX Part(),
        saveInto: a!save(
            ri!estimate[W#ADX Estimate.part],
            append(
                ri!estimate[W#ADX Estimate.part],
                save!value
            )
        )
    )
},

```

```

        shape: "SEMI_ROUNDED",
        padding: "MORE",
        marginBelow: "STANDARD",
        style: "NONE"
    )
}
)
},
primaryButtons: a!buttonWidget(
    label: "Create",
    submit: true,
    validate: true,
    style: "SOLID",
    showWhen: fv!isLastStep
),
secondaryButtons: a!buttonWidget(
    label: "Cancel",
    submit: true,
    validate: false,
    saveInto: ri!cancel,
    style: "LINK",
    value: true
),
style: "DOT_VERTICAL",
showTitleBarDivider: false,
contentsWidth: "FULL"
)
)

```